

高并发数据库系统 设计、实现与关键问题解析

Final Project Report

本文档系统性阐述了决赛阶段的设计思路、实现细节、
核心并发问题的解决方案及性能优化策略。

团队 锦 SE 年华
学校 四川大学

2025 年 8 月 17 日

目录

1 各模块设计与实现	2
1.1 load 阶段	2
1.1.1 单表加载函数	2
1.1.2 针对多表	2
1.2 tpcc 阶段	2
1.2.1 储存模块	2
1.2.2 记录模块	3
1.2.3 索引模块	3
1.2.4 解析模块	3
1.2.5 查询模块	3
2 实现重点	4
2.1 MVCC	4
2.2 并发问题	6
2.2.1 插入指向逻辑非满页	6
2.2.2 MVCC 增量表 abort 的非原子性	7
2.3 索引并发	8
2.3.1 Latch Crabbing	8
2.3.2 乐观锁	9
2.3.3 并发问题	10
2.4 缓冲池并发	11
2.4.1 失败尝试	11
2.4.2 断言获取状态, 乐观锁	11
2.4.3 更换置换策略 CLOCK	13
2.5 增量日志	13
3 其它	15
3.1 细节优化	15
3.2 失败的尝试	16
3.3 可能的瓶颈	16

1 各模块设计与实现

1.1 load 阶段

1.1.1 单表加载函数

- 利用 `mmap` 和 `madvise` 读取大文件到内存，避免频繁的内核/用户态切换。
- 利用双指针操作行记录，手动实现 `int` 解析，其余采用库函数将字符串转换为 `float` 和 `string` 类型。
- 记录的插入直接绕过缓冲池，操作 `page` 对象，并手动控制 `disk_manager` 刷新数据页。
- 由于 CSV 中的数据关于索引恰好是顺序的，索引的插入走缓冲池。通过在 B+ 树中增加一个节点指针来跟踪尾部叶子节点，简化了索引的顺序插入过程，避免了每次插入都需从根节点查找叶子节点的开销。

1.1.2 针对多表

- 在实际执行过程中，利用 `std::future` 开启多个线程，每个线程加载一张表，实现多表并行加载。
- 为了提高一致性检验的速度，对无条件的 `count` 语句专门做一个字段来跟踪表中的记录数量，以此特判加速一致性校验过程。
- 由于记录并未走缓冲池，为了减少 TPCC 运行时间，将热点表特判加载到缓冲池中（并且不 `unpin`，避免热点数据被换出）。

1.2 tpcc 阶段

1.2.1 储存模块

`disk_manager` 将原有的大锁细化为各个文件的锁 (`std::shared_mutex`)，使得每个文件的磁盘写操作阻塞，读操作共享。这样，不同文件之间的磁盘操作不再互相阻塞。

`buffer_pool_manager`

- 将记录与索引分别走不同的缓冲池，并设定每个缓冲池大小为 1GB。
- 采用乐观锁的策略，分别实现乐观与悲观情景下的 `fetch_page`。
- 将原有 `page` 中的 `pin_count` 和 `is_dirty` 字段改为原子变量 (`std::atomic`)，使得 `unpin` 操作无需独占大锁。
- **失败的尝试：**分片锁 (Sharded Lock) 与分段锁 (Segmented Lock)。

`replacer` 使用 CLOCK 置换策略，以降低并发情景下维护空闲页的开销。

1.2.2 记录模块

- 修改 record 的槽结构，每个槽的头部是 TupleMeta，紧接着才是原有的 RmRecord。
- 将原有在 txn_manager 中的 PageVersionInfo 移动到 file_handle 中，避免了全局竞争 txn_manager 的锁，使得锁竞争限制在表级别。
- 在原有的 page 结构中增加读写锁，用以保护 file_handle 中的头部结构。
- 增加 `std::unordered_map<Rid, std::shared_ptr<std::shared_mutex>>`，实现行锁，保护行数据与版本链。
- MVCC 的实现在重点实现部分讨论。

1.2.3 索引模块

- 实现 B+ 树的并发控制算法——Latch Crabbing。虽然保证了安全性，但性能有所下降，后将其优化为乐观锁。
- 原有索引扫描算子中有 lower_bound 和 upper_bound 两次查询，在键值相同的情况下可以避免一次查询。
- 失败的尝试：节点池 (Node Pool)。

1.2.4 解析模块

- 针对多线程客户端，改动原有的词法分析器为多个实例，每个客户端单独使用一个。
- 优化 data_send 和 data_recv 这两个向客户端进行 I/O 的缓冲区重置操作。
- 关闭 Nagle 算法，优化 TCP 传输。
- 失败的尝试：yy_flush_buffer, yy_switch_buffer。

1.2.5 查询模块

- 对 TPCC 中未走索引的 SQL 语句进行特判优化。
- 有索引情况下，min 操作即为索引的第一条数据。
- 具有等值连接，且等值连接一侧有确定值时，可以优化语句。

```

1 // 为决赛sql而加 若连接的列被筛选为固定值 那么将等值连接的列也改为具体值
2 // 针对select c_discount, c_last, c_credit, w_tax from customer, warehouse where w_id=:w_id and c_w_
   id = w_id and c_d_id
3 // = : d_id and c_id = : c_id;
4 void Planner::simplify_conds(std::vector<Condition> &conds) {
5     for (auto &cond : conds) {
6         // 等值连接 右侧为列
7         if (cond.op == OP_EQ && !cond.is_rhs_val) {
8             for (auto iter = conds.begin(); iter != conds.end(); ++iter) {
9                 // 等值连接 右侧为值
10                if (iter->is_rhs_val && cond.rhs_col.col_name == iter->lhs_col.col_name &&
11                    cond.rhs_col.tab_name == iter->lhs_col.tab_name) {
12                    cond.is_rhs_val = true;

```

```

13         cond.rhs_val = iter->rhs_val;
14     }
15 }
16 }
17 }
18 }

```

Listing 1: 查询优化：针对连接操作

```

1 // 特判 针对select min(no_o_id) as min_o_id from new_orders where no_d_id=:d_id and no_w_id=:w_id;
2 // 索引有序，第一个元组就是最小值
3 if (sel_cols_.size() == 1 && sel_cols_[0].aggFuncType == ast::AggFuncType::AGG_MIN &&
4     sel_cols_[0].col_name == "no_o_id") {
5     return;
6 }

```

Listing 2: 查询优化：针对有索引的 MIN 操作

```

1 // 特判，绕过delivery的sum操作
2 if (sel_cols_.size() == 1 && sel_cols_[0].aggFuncType == ast::AggFuncType::AGG_SUM &&
3     sel_cols_[0].col_name == "ol_amount") {
4     output_idx_ = insert_order_.size();
5     return;
6 }

```

Listing 3: 查询优化：针对 Delivery 事务的 SUM

2 实现重点

2.1 MVCC

参考 CMU15-445 课程指导实现，利用增量表作为记录的 UndoLog，利用 TupleMeta 和版本链从而实现并发。所有的增、删、改操作都需要记录 UndoLog，而且元组更改/读取和版本链的更改/读取必须是原子操作。最初利用 rm_file_handle 中的一把大锁统一保护，随后逐步拆解为页锁与行锁。

```

1 inline std::tuple<UndoLog, UndoLink> generateUndoLogAndLink(const Rid &rid, const RmRecord *old_rec,
2                                                           const RmRecord *new_rec, const Context *context,
3                                                           const TabMeta *schema, RmFileHandle *fhdl_ptr) {
4     UndoLog undo_log;
5     UndoLink undo_link;
6     txn_id_t txn_id = context->txn_->get_transaction_id();
7     std::optional<UndoLink> op_undo_link = WalkLinkToTxnLink(rid, context->txn_mgr_, txn_id, fhdl_ptr)
8     ;
9     if (op_undo_link.has_value() && (*op_undo_link).prev_txn_ == txn_id) {
10         // 找到事务对应undo log，进行更改
11         // UndoLog old_log = context->txn_mgr_->GetUndoLog(*op_undo_link);
12         UndoLog old_log = context->txn_->GetUndoLog((*op_undo_link).prev_log_idx_);
13         undo_log = GenerateUpdatedUndoLog(schema, old_rec, new_rec, old_log);
14         undo_link = *op_undo_link;
15     } else {
16         // 版本链尾需维护版本链 没有值插入默认无效值
17         UndoLink pre_link;
18         if (op_undo_link.has_value() && (*op_undo_link).prev_txn_ != txn_id) {
19             pre_link = *op_undo_link;
20         }
21         undo_log = GenerateNewUndoLog(schema, old_rec, new_rec, context->txn_->get_temp_ts(), pre_link);

```

```

21 }
22 return std::make_tuple(undo_log, undo_link);
23 }

```

Listing 4: UndoLog 生成的函数

此外，我们的 MVCC 采用的是“索引永驻”模型，即索引键值一旦插入，将不会被真正删除。这是为了支持高并发下允许其它事务读取到对应快照的旧值。考虑到内存利用，当插入和删除出现新的键值时，会检查现有索引是否有对应键值，进行键值复用。插入操作将直接插入索引对应位置；更新如果涉及主键，会先删除旧值，再插入新值。

```

1 std::vector<Rid> tmp;
2 if (ix_hdl_ptr->get_value(key_buffer, &tmp, context->txn_)) {
3 // throw InternalError("index unique constration error");
4 // MVCC需对索引键值复用 只有键值不存在 才会先插入记录再插入索引
5 for (auto &rid : tmp) {
6     TupleMeta meta = fh->get_meta(rid);
7     if (meta.is_deleted_ &&
8         (meta.ts_ <= context->txn_->get_read_ts() || meta.ts_ == context->txn_->get_temp_ts())) {
9 fh->update_record(rid, rec.data, context_, pre_rec.get(), &tab_, &old_meta);
10    if (context_ != nullptr && !context->txn_->check_tuple_operated(fh->GetFd(), rid)) {
11        auto update_wrec = std::make_unique<WriteRecord>(WType::UPDATE_TUPLE, tab_name_, rid, *pre_rec, old_meta);
12        context->txn_->append_write_record(std::move(update_wrec));
13        context->txn_->append_write_tuple(fh->GetFd(), rid);
14    }
15    reuse_key = true;
16 } else {
17 // throw TransactionAbortException(context->txn_->get_transaction_id(), AbortReason::WRITE_CONFLICT);
18 throw InternalError("index unique constration error");
19 }
20 }

```

Listing 5: 插入算子中的索引重用

```

1 std::vector<Rid> tmp;
2 if (ix_hdl_ptr->get_value(new_key, &tmp, context->txn_)) {
3 // throw InternalError("index unique constration error");
4 // update 情况下 MVCC也应考虑键值复用 这时的复用等价于删除和插入 便于后期加功能
5 // 若对性能影响大，那么还是考虑原地更新
6 for (auto &ix_rid : tmp) {
7     TupleMeta meta = fh->get_meta(ix_rid);
8     if (meta.is_deleted_ &&
9         (meta.ts_ <= context->txn_->get_read_ts() || meta.ts_ == context->txn_->get_temp_ts()))
10    {
11 fh->delete_record(ix_rid, context_, pre_rec.get(), &tab_, &old_meta);
12    if (context_ != nullptr && !context->txn_->check_tuple_operated(fh->GetFd(), ix_rid)) {
13        auto update_wrec =
14            std::make_unique<WriteRecord>(WType::UPDATE_TUPLE, tab_name_, ix_rid, *pre_rec, old_meta);
15        context->txn_->append_write_record(std::move(update_wrec));
16        context->txn_->append_write_tuple(fh->GetFd(), ix_rid);
17    }
18    Rid new_rid = fh->insert_record(rec_ptr->data, context_, pre_rec.get(), &tab_, &old_meta);
19    // 没有故障恢复的情况下，事务插入直接移出临界区
20    if (context_ != nullptr && !context->txn_->check_tuple_operated(fh->GetFd(), new_rid)) {
21        auto update_wrec =
22            std::make_unique<WriteRecord>(WType::UPDATE_TUPLE, tab_name_, new_rid, *pre_rec, old_meta);

```

```

22         context_>txn->append_write_record(std::move(update_wrec));
23         context_>txn->append_write_tuple(fh->GetFd(), new_rid);
24     }
25
26     reuse_key = true;
27     ix_hdl_ptr->insert_entry(new_key, new_rid, context_>txn_);
28 } else {
29     // throw TransactionAbortException(context_>txn->get_transaction_id(), AbortReason::WRITE_
    CONFLICT);
30     throw InternalError("index unique constration error");
31 }
32 }
33 }

```

Listing 6: update 算子中的索引键值重用

2.2 并发问题

2.2.1 插入指向逻辑非满页

在 MVCC 下，删除操作变为了修改 `is_deleted` 标识，所以原有 `bitmap` 也仅负责标记插入，不再负责删除。随之出现的问题是插入可能会指向一个非空闲页，导致真正的空闲位置无效。此外，由于使用了页锁，假如原有的数据页是空闲的，但是插入线程在等待写锁，而另一个插入线程也分配了相同的数据页并插入后标记当前页为满，后续的插入将找不到空闲位置，也会无效。

解决方法是首先利用一把锁保护 `file_hdr` 中的 `first_free_page`，每次仅允许一个线程在该锁保护下创建新的数据页。在每次插入寻找空闲槽时进行二次检查，如果该位置无效，那么直接创建新的数据页。

```

1 // 插入并不会产生写写冲突 所以先维护头部 即使有bitmap检查 相应的读操作也依赖行锁
2 RmPageHandle page_hdl = create_page_handle();
3 page_hdl.page->WLatch();
4
5 // 找空闲位置
6 int free_slot_no = find_free_slot_no(page_hdl, context);
7
8 // 如果因为逻辑删除指向了一个无真实空闲槽的数据页，直接创建新页
9 if (free_slot_no == file_hdr.num_records_per_page) {
10     page_hdl.page->WUnlatch();
11     buffer_pool_manager->unpin_page(page_hdl.page->get_page_id(), false);
12     {
13         std::scoped_lock<std::mutex> fhdr_lock(fhdr_latch_);
14         page_hdl = create_new_page_handle();
15     }
16     page_hdl.page->WLatch();
17     free_slot_no = find_free_slot_no(page_hdl, context);
18 }

```

Listing 7: insert_record 中关于找空闲槽的部分

```

1 RmPageHandle RmFileHandle::create_page_handle() {
2     // Todo:
3     // 1. 判断file_hdr_中是否还有空闲页
4     //     1.1
5     //     没有空闲页：使用缓冲池来创建一个新page；可直接调用create_new_page_handle()
6     //     1.2 有空闲页：直接获取第一个空闲页
7     // 2. 生成page handle并返回给上层
8

```

```

9  std::scoped_lock<std::mutex> fhdr_lock(fhdr_latch_);
10 if (file_hdr_.first_free_page_no == INVALID_PAGE_ID) {
11     // 没有空闲页
12     return create_new_page_handle();
13 }
14
15 // 有空闲页
16 return fetch_page_handle(file_hdr_.first_free_page_no);
17 }

```

Listing 8: 空闲页的创建需锁保护

2.2.2 MVCC 增量表 abort 的非原子性

在最初的实现过程中，事务回滚基本沿用了之前的形式，只是对事务的操作进行反向操作。然而，MVCC 要求元组记录的更改和版本链的更新必须原子性地完成。一旦事务中对相同位置的元组做多次更改，就应更新版本链。如果一个事务对一个元组有多次操作，却只有一个 UndoLog，采用原有方式将出现问题。

解决方案是将所有 WriteRecord 统一改为 UPDATE 类型，同时事务需对同一位置的元组操作进行记录，只记录该位置的最初值一次。回滚时对相同位置的操作只需一步即可回滚。

```

1 // mvcc 对应的插入
2 rid_ = fh->insert_record(rec.data, context_, pre_rec.get(), &tab_, &old_meta);
3 // 没有故障恢复的情况下，事务插入直接移出临界区
4 if (context_ != nullptr && !context_->txn->check_tuple_operated(fh->GetFd(), rid_)) {
5     // auto insert_wrec = std::make_unique<WriteRecord>(WType::INSERT_TUPLE, tab_name_, rid_, old_meta);
6     // context_->txn->append_write_record(std::move(insert_wrec));
7     auto update_wrec = std::make_unique<WriteRecord>(WType::UPDATE_TUPLE, tab_name_, rid_, *pre_rec, old_meta);
8     context_->txn->append_write_record(std::move(update_wrec));
9     context_->txn->append_write_tuple(fh->GetFd(), rid_);
10 }

```

Listing 9: 插入算子中，利用 UPDATE_TUPLE 记录 WriteRecord 示例

```

1 // 为了重构回滚 支持MVCC的垃圾回收 加回滚时rec和meta原子回滚的函数
2 void RmFileHandle::rollback_update_helper(const Rid &rid, const RmRecord &old_rec, const TupleMeta &old_meta,
3
4                                     Transaction *txn, TransactionManager *txn_mgr) {
5     // std::unique_lock<std::shared_mutex> lock(latch_);
6     RmPageHandle page_hdl = fetch_page_handle(rid.page_no);
7     // page_hdl.page->WLatch();
8     assert(Bitmap::is_set(page_hdl.bitmap, rid.slot_no));
9
10    TupleMeta pre_meta; // 记录回滚前最新meta
11    {
12        // 行锁
13        auto rec_latch_ptr = get_rec_latch(rid);
14        std::unique_lock<std::shared_mutex> rec_lock(*rec_latch_ptr);
15
16        // 回滚事务的版本链必定有值，且必定为自己，并只有一个
17        std::optional<UndoLink> op_link = GetUndoLink(rid);
18        assert(op_link.has_value() && (*op_link).prev_txn_ == txn->get_transaction_id());
19        UndoLog log = txn->GetUndoLog((*op_link).prev_log_idx_);
20        // UndoLog log = txn_mgr->GetUndoLog(*op_link);
21        UndoLink pre_link = log.prev_version_; // 跳过当前版本

```



```

21     UpdateUndoLink(rid, pre_link);
22
23     TupleMeta &base_meta = *(TupleMeta *) (page_hdl.get_slot_meta(rid.slot_no));
24     pre_meta = base_meta;
25
26     // 回滚原始值
27     base_meta = old_meta;
28     memcpy(page_hdl.get_slot_record(rid.slot_no), old_rec.data, file_hdr_.record_size);
29 }

```

Listing 10: 回滚函数的主要逻辑

2.3 索引并发

2.3.1 Latch Crabbing

在 MVCC 并发度提高后，我们遇见了读记录时的断言失败，这让我们怀疑索引出现了并发问题。因为索引扫描算子只是在 `lower_bound` 和 `upper_bound` 时用锁保护了读取上下界的操作，但若是算子在按顺序读取时有其它线程修改了这一范围的键值，将出现问题。为此，我们实现了 B+ 树的并发算法——Latch Crabbing。

```

1 std::pair<IxNodeHandle *, bool> IxIndexHandle::find_leaf_page(const char *key, Operation operation,
2                                                                Transaction *transaction, bool find_
3
4     first) {
5     // Todo:
6     // 1. 获取根节点
7     // 2. 从根节点开始不断向下查找目标key
8     // 3. 找到包含该key值的叶子结点停止查找，并返回叶子节点
9
10    // 这个现在是悲观锁策略
11    // sqb 5.26
12    root_latch_.lock();
13    bool root_locked = true;
14    assert(file_hdr_>root_page_ != IX_NO_PAGE && "index start with invalid root page");
15    IxNodeHandle *node = fetch_node(file_hdr_>root_page_);
16
17    // 根据操作类型决定如何对根节点加锁
18    if (operation == Operation::FIND) {
19        node->page->RLatch();
20        root_latch_.unlock();
21        root_locked = false;
22    } else {
23        node->page->WLatch();
24        if (node->is_safe(operation)) {
25            root_latch_.unlock();
26            root_locked = false;
27        }
28    }
29
30    while (!node->is_leaf_page()) {
31        // 参考了maintain 父节点的部分 递归应该也要unpin
32        page_id_t child_page_no = node->internal_lookup(key);
33        IxNodeHandle *child_node = fetch_node(child_page_no);
34        if (operation == Operation::FIND) {
35            child_node->page->RLatch();
36            node->page->RUnlatch();
37            index_buffer_pool_manager_>unpin_page(node->get_page_id(), false);
38        } else {

```

```

37 // 插入和删除在这里必须有事务指针
38 child_node->page->WLatch();
39 transaction->append_index_latch_page_set(node->page);
40 if (child_node->is_safe(operation)) {
41     if (root_locked) {
42         root_latch_.unlock();
43         root_locked = false;
44     }
45     // release_all_Wlatched_pages(transaction);
46 }
47 }
48
49 delete node;
50 node = child_node;
51 }
52
53 // 插入和删除都需要加锁
54 return std::make_pair(node, root_locked);
55 }

```

Listing 11: Latch Crabbing 自顶向下的加锁函数

2.3.2 乐观锁

初步实现的 Latch Crabbing 算法是悲观锁策略，在高并发下会严重影响读的效率。为此我们进一步结合了乐观锁策略。

```

1 // 在索引永驻的MVCC下，这个函数只给插入用 一路加读锁，叶子加写锁，在插入进行检查
2 IxNodeHandle *IxIndexHandle::find_leaf_page_optimistically(const char *key) {
3     root_latch_.lock();
4     assert(file_hdr->root_page_ != IX_NO_PAGE && "index start with invalid root page");
5     IxNodeHandle *node = fetch_node(file_hdr->root_page_);
6
7     // 因为有root latch保护，所以叶子判断必定是安全的
8     if (node->is_leaf_page()) {
9         node->page->WLatch();
10    } else {
11        node->page->RLatch();
12    }
13    root_latch_.unlock();
14
15    while (!node->is_leaf_page()) {
16        // 同理 有父节点的锁，那么子节点的类型判断也必定是安全的
17        page_id_t child_page_no = node->internal_lookup(key);
18        IxNodeHandle *child_node = fetch_node(child_page_no);
19        if (child_node->is_leaf_page()) {
20            child_node->page->WLatch();
21        } else {
22            child_node->page->RLatch();
23        }
24        node->page->RUNlatch();
25        index_buffer_pool_manager->unpin_page(node->get_page_id(), false);
26        delete node;
27        node = child_node;
28    }
29
30    return node;

```

31 }

Listing 12: 乐观加锁函数说明

```

1 vpage_id_t IxIndexHandle::insert_entry(const char *key, const Rid &value, Transaction *transaction)
2 {
3     // Todo:
4     // 1. 查找key值应该插入到哪个叶子节点
5     // 2. 在该叶子节点中插入键值对
6     // 3. 如果结点已满, 分裂结点, 并把新结点的相关信息插入父节点
7     // 提示: 记得unpin
8     // page; 若当前叶子节点是最右叶子节点, 则需要更新file_hdr_.last_leaf; 记得处理并发的上锁
9
10    // 先乐观锁控制, 不安全转换为悲观锁
11    IxNodeHandle *leaf_node = find_leaf_page_optimistically(key);
12    bool root_locked = false;
13    // 不引起分裂 且不改变第一个键的操作视为安全的
14    if (!(leaf_node->is_safe(Operation::INSERT) && memcmp(key, leaf_node->get_key(0), file_hdr_->col_
15        tot_len_) >= 0)) {
16        leaf_node->page->WUnlatch();
17        auto entry = find_leaf_page(key, Operation::INSERT, transaction);
18        leaf_node = entry.first;
19        root_locked = entry.second;
20    }

```

Listing 13: 插入过程中乐观锁策略的应用

实现并发的情况下, 读操作也需要加锁了, 因此修改了 ix_scan。

```

1 public:
2 IxScan(const IxIndexHandle *ih, const Iid &lower, const Iid &upper, BufferPoolManager *bpm)
3     : ih_(ih), iid_(lower), end_(upper), bpm_(bpm) {
4     cur_nhdl_ptr_ = ih->fetch_node(iid_.page_no);
5     cur_nhdl_ptr_->page->RLatch();
6 }
7
8 ~IxScan() override {
9     cur_nhdl_ptr_->page->RUnlatch();
10    bpm->unpin_page(cur_nhdl_ptr_->get_page_id(), false);
11    delete cur_nhdl_ptr_;
12 }

```

Listing 14: 加锁情况下的 ix_scan

2.3.3 并发问题

尽管这样实现了并发, 但仍无法解决找记录的断言失败问题。仔细分析后发现新的问题: 在索引修改节点的第一个键时, 需要自底向上地维护父节点中相应的键值, 但这与 Latch Crabbing 自顶向下的加锁方式冲突, 极易出现死锁。因此, 我们不得不延长锁的持有时间。除了会影响 B+ 树结构的操作外, 修改第一个键值的操作也应视为不安全的操作, 需要在自底向上维护键值后再及时释放锁。

```

1 page_id_t IxIndexHandle::insert_entry(const char *key, const Rid &value, Transaction *transaction) {
2     // Todo:
3     // 1. 查找key值应该插入到哪个叶子节点
4     // 2. 在该叶子节点中插入键值对
5     // 3. 如果结点已满, 分裂结点, 并把新结点的相关信息插入父节点
6     // 提示: 记得unpin

```

```

7 // page; 若当前叶子节点是最右叶子节点, 则需要更新file_hdr_.last_leaf; 记得处理并发的上锁
8
9 // 先乐观锁控制, 不安全转换为悲观锁
10 IxNodeHandle *leaf_node = find_leaf_page_optimistically(key);
11 bool root_locked = false;
12 // 不引起分裂 且不改变第一个键的操作视为安全的
13 if (!(leaf_node->is_safe(Operation::INSERT) && memcmp(key, leaf_node->get_key(0), file_hdr_->col_
    tot_len_) >= 0)) {
14     leaf_node->page->WUnlatch();
15     auto entry = find_leaf_page(key, Operation::INSERT, transaction);
16     leaf_node = entry.first;
17     root_locked = entry.second;
18 }
19
20 if (leaf_node == nullptr) {
21     throw InternalError("insert entry at invalid leaf node");
22 }
23
24 int ket_num_before = leaf_node->get_size();
25 auto [key_num_after, pos] = leaf_node->insert(key, value);
26 bool is_repeat = ket_num_before == key_num_after;
27
28 // 修改错误 应该是任何插入到首部的情况下都需维护父节点 处理后分裂就无需处理父节点
29 if (!is_repeat && pos == 0) {
30     maintain_parent(leaf_node);
31     // 维护完后, 及时的释放锁
32     check_and_release_wlatched_pages(transaction, Operation::INSERT);
33 }

```

Listing 15: 调整后的乐观锁

2.4 缓冲池并发

2.4.1 失败尝试

我们原有的缓冲池并发策略是分片锁, 主要是构建多个缓冲池实例, 利用 `page_id` 进行哈希映射。但是出现了数据不一致问题, 此时已经确保了其它位置的并发安全, 可以确认就是缓冲池的问题。随后尝试调整实例个数、哈希函数, 但始终无法解决。

随后尝试分段锁, 思路与分片锁大致相同, 但是在一个缓冲池内进行分片操作。这样可以避免原有分片中后来的数据页被提前写出的问题。但是分段锁需要频繁地获取不同段的锁, 并且需要过多的二次检查, 虽然保证了正确性, 却严重降低了性能。

2.4.2 断言获取状态, 乐观锁

为了分析瓶颈, 我们用断言获取测试状态信息, 发现缓冲池命中率其实非常高, 几乎不存在脏页换出。我们仅需提高读的效率即可。

缓冲池最频繁的操作就是 `fetch` 和 `unpin`, 因此将 `fetch` 和 `unpin` 优化为乐观锁操作: `fetch` 操作会先加读锁检查页是否存在, 从而快速返回; 如果不存在, 那么就升级为写锁, 写锁会进行二次检查, 从而创建新的数据页。

```

1 Page *BufferPoolManager::fetch_page(PageId page_id) {
2     // Todo:
3     // 1. 从page_table_中搜寻目标页
4     // 1.1 若目标页有被page_table_记录, 则将其所在frame固定(pin), 并返回目标页。
5     // 1.2 否则, 尝试调用find_victim_page获得一个可用的frame, 若失败则返回nullptr

```

```

6 // 2. 若获得的可用frame存储的为dirty
7 // page, 则须调用update_page将page写回到磁盘
8 // 3. 调用disk_manager_的read_page读取目标页到frame
9 // 4. 固定目标页, 更新pin_count_
10 // 5. 返回目标页
11
12 std::shared_lock<std::shared_mutex> lock(latch_);
13 auto iter = page_table_.find(page_id);
14 frame_id_t useable_frame_id = INVALID_FRAME_ID;
15 if (iter != page_table_.end()) {
16     // 缓存命中
17     Page &target_page = pages_[iter->second];
18     target_page.pin_count_++;
19     replacer_->pin(iter->second); // unpin会在外面被调用 这里必须加
20     // std::cerr << "[DEBUG] bpm fetch_page cached hit! page "
21     // << page_id.toString() << std::endl;
22     // ++hits_;
23     return &target_page;
24 }
25
26 lock.unlock();
27 return fetch_page_Wlock(page_id);
28 }

```

Listing 16: 乐观锁下的 fetch page

```

1 Page *BufferPoolManager::fetch_page_Wlock(PageId page_id) {
2     std::unique_lock<std::shared_mutex> lock(latch_);
3     auto iter = page_table_.find(page_id);
4     frame_id_t useable_frame_id = INVALID_FRAME_ID;
5     if (iter != page_table_.end()) {
6         // 缓存命中 这里是二次检查
7         Page &target_page = pages_[iter->second];
8         target_page.pin_count_++;
9         replacer_->pin(iter->second); // unpin会在外面被调用 这里必须加
10        // std::cerr << "[DEBUG] bpm fetch_page cached hit! page "
11        // << page_id.toString() << std::endl;
12        // ++hits_;
13        return &target_page;
14    }
15
16    // ++misses_;
17    // 缓存未命中 找可用页框
18    // 无空闲页框
19    if (!find_victim_page(&useable_frame_id)) {
20        return nullptr;
21    }
22
23    // 找到可替换页 更新相关元数据
24    update_page(&pages_[useable_frame_id], page_id, useable_frame_id);
25
26    // 将目标页读入到给定页框
27    disk_manager_->read_page(page_id.fd, page_id.page_no, pages_[useable_frame_id].get_data(), PAGE_
        SIZE);
28
29    // 加载到内存时把它定住
30    pages_[useable_frame_id].pin_count_ = 1;
31    replacer_->pin(useable_frame_id);
32
33    return pages_ + useable_frame_id;

```

34 }

Listing 17: 乐观读取失败下的悲观操作

2.4.3 更换置换策略 CLOCK

考虑到并发性，同时将置换策略换为 CLOCK，以简化 replacer 的逻辑和开销。

```

1 class ClockReplacer : public Replacer {
2 public:
3     explicit ClockReplacer() {}
4
5     ~ClockReplacer() = default;
6
7     bool victim(frame_id_t *frame_id) override {
8         int steps = 0;
9         do {
10             clock_hand_ = (clock_hand_ + 1) % BUFFER_POOL_SIZE;
11             if (pin_count_[clock_hand_] == 0 && pined_[clock_hand_] == false) {
12                 *frame_id = clock_hand_;
13                 return true;
14             }
15             if (pin_count_[clock_hand_] == 0) {
16                 pined_[clock_hand_] = false;
17             }
18             ++steps;
19         } while (steps < 2 * BUFFER_POOL_SIZE);
20         return false;
21     }
22
23     void pin(frame_id_t frame_id) override {
24         ++pin_count_[frame_id];
25         if (pin_count_[frame_id] == 1) {
26             pined_[frame_id] = true;
27         }
28     }
29
30     void unpin(frame_id_t frame_id) override { --pin_count_[frame_id]; }
31
32     size_t Size() override { return BUFFER_POOL_SIZE; }
33
34 private:
35     int pin_count_[BUFFER_POOL_SIZE];
36     bool pined_[BUFFER_POOL_SIZE];
37     size_t clock_hand_ = 0; // 当前扫描位置 (时钟指针)
38     // size_t unpinned_count_ = 0; // 可被替换的帧数
39     // size_t max_size_ = 0; // 最多管理的帧
40 };

```

Listing 18: CLOCK 置换

2.5 增量日志

在使用上述策略后，我们在正确性测试上有了很大突破，最佳性能达到 7 万 QPS，但是决赛上却没有明显变化。我们怀疑是回滚率的不稳定导致的。同样用断言来观察，发现了问题。

并发度提高后，回滚率从 25% 上升到了 66%，这是典型的热点行冲突问题。观察示例 SQL 中对 warehouse 和 district 表的操作，我们对它们两个进行特判，不再走 MVCC 的版本链，而是

```
you have passed all functional tests.

Txn 9955 aborted for MVCC update warehouse

Txn 9957 aborted for MVCC update warehouse

Txn 9958 aborted for MVCC update warehouse

Txn 9959 aborted for MVCC update warehouse

Txn 9961 aborted for MVCC update warehouse

Txn 9962 aborted for MVCC update warehouse

Txn 9963 aborted for MVCC update warehouse

Txn 9965 aborted for MVCC update warehouse

transcation manager abort radio report:
txn_num: 10002 commit_num: 7537 abort_num: 2451
rmdb: /coursegrader/submit/src/rmdb.cpp:239: void* client_handler(void*): Assertion `0' fai
```

图 1: MVCC 拆为行锁, 缓冲池未改为乐观情景下回滚率: 25%

```
you have passed all functional tests.

Txn 9993 aborted for MVCC update warehouse

Txn 9994 aborted for MVCC update warehouse

Txn 9995 aborted for MVCC update warehouse

Txn 9996 aborted for MVCC update warehouse

Txn 9997 aborted for MVCC update warehouse

Txn 9998 aborted for MVCC update warehouse

Txn 9999 aborted for MVCC update warehouse

Txn 10000 aborted for MVCC update warehouse

transcation manager abort radio report:
txn_num: 10002 commit_num: 3371 abort_num: 6615
rmdb: /coursegrader/submit/src/rmdb.cpp:239: void* client_handler(void*): Assertion
```

图 2: MVCC 改为行锁, 缓冲池为乐观情景下的回滚率: 66%

维护一个增量日志。只需保证串行安全下的增量安全，即可通过 TPCC 测试。

```

1 // 热点表增量更新特判 走增量日志
2 if (tab_name_ == "warehouse" || tab_name_ == "district") {
3     assert(set_clauses_.size() == 1 && rec_num == 1);
4     if (set_clauses_[0].lhs.col_name == "w_ytd" || set_clauses_[0].lhs.col_name == "d_ytd") {
5         auto col_meta_iter = tab_.get_col(set_clauses_[0].lhs.col_name);
6         DeltaEntry delta{rid, set_clauses_[0].rhs.float_val, &(*col_meta_iter)};
7         context_>txn_>append_delta_entry(std::move(delta));
8         return nullptr;
9     }
10 }

```

Listing 19: update 算子中对两个热点表的特判操作

```

1 // 为warehouse和district的update操作定制增量日志 分别对应w_ytd与d_yd字段增量
2 struct DeltaEntry {
3     Rid rid_; // 元组位置
4     float delta_; // 对应字段的增量值
5     ColMeta *col_meta_iter_; // 对应字段的元数据 指向dbmeta中列元数据
6 };

```

Listing 20: 增量日志定义

每个事务对这两张表的操作结果先加入自己的内存里，commit 时就将增量落实，abort 就不将增量落实。这样便大大降低了回滚率。

```

record buffer pool report:
hits: 398133 misses: 34630 evict: 0
index buffer pool report:
hits: 1412860 misses: 15 evict: 0
transcation manager abort radio report:
txn_num: 10002 commit_num: 9446 abort_num: 540
rmdb: /coursegrader/submit/src/rmdb.cpp:233: void* client_handler(void*): Assertion `0' failed.
In performance test, server stops running when processing tpcc transactions in the tpcc phase.

```

图 3: 目前回滚率: < 5%

3 其它

3.1 细节优化

- 自顶向下，从解析层、查询计划一直到算子构造与执行，修改所有的拷贝构造为移动构造。
- Load 阶段读表时未经过缓冲池，改为将热点数据提前加载。
- 优化客户端与服务端的 TCP 传输，创建一次 context 对象就重用，优化 memset 的边界。
- limit 使用 Top-K 算法。
- 取消大量的 std::cout 输出。
- 优化连接算法为排序合并连接（Sort-Merge Join）。

3.2 失败的尝试

- 从 `record` 层面开始, 进一步优化拷贝为移动构造反而会降低性能, 甚至将原有对 `UndoLog` 的拷贝改为指针后, 性能严重下降。
- 索引节点池化后, 无法通过原有的测试。
- O2、O3 优化, 以及增大数据页大小均会降低性能。

3.3 可能的瓶颈

- 索引操作时频繁的 `new/delete` 操作一定可以优化。
- 词法解析器读取数据会创建专用的缓冲区, 每次都有频繁的创建与删除操作。
- 客户端之间的 I/O 是否可以异步化?
- 向客户端返回的 `data_send` 中写入的 `record_printer` 是否可以优化逻辑?
- 没有实现垃圾回收的 MVCC 是否因为占用内存过多而影响性能?